

Functions

Functions are reusable blocks of code that perform a specific task. They help in organizing code, improving readability, and reducing redundancy. Functions help break down complex programs into smaller, more manageable pieces, making code easier to read, write, and maintain. Python treats functions as first-class objects, meaning they can be assigned to variables, passed to other functions and returned as values.

There are three types of functions in Python:

- Built-in functions, such as `help()` to ask for help, `min()` to get the minimum value, `print()` to print an object to the terminal.
- User-Defined Functions (UDFs), which are functions that users create to help them out; And
- Anonymous functions, which are also called lambda functions because they are not declared with the standard `def` keyword.

Defining a Function

Defining a function involves creating a reusable block of code that performs a specific task. Functions are defined using the `def` keyword, followed by the function name, a pair of parentheses `()`, and a colon `:`. The block of code inside the function is indented and executed when the function is called.

Syntax

```
def function_name(parameters):  
    # Function body  
    # Code to execute
```

- **def:** Keyword used to define a function.
- **function_name:** Name of the function (follows the same rules as variable names).
- **Parameters:** Optional inputs to the function (enclosed in parentheses).
- **Function body:** The indented block of code that defines what the function does.
- **Return Value:** Functions can return a value using the `return` statement.

If no `return` statement is provided, the function returns **None**.

Calling a Function

Calling a function means executing the code defined within the function. Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can call it by using the function name itself. If the function requires any parameters, they should be passed within parentheses. If the function doesn't require any parameters, the parentheses should be left empty.

Syntax

```
function_name(arguments)
```

- **function_name:** Name of the function to call.
- **arguments:** Values passed to the function (if it accepts parameters).
- If the function has parameters, provide the required arguments in the correct order.
- Arguments can be positional, keyword, or default.
- **return value:** If the function returns a value, you can store it in a variable or use it directly.

Pass by Reference vs Value

call by value – When a variable is passed to a function while calling, the value of actual arguments is copied to the variables representing the formal arguments. Thus, any changes in formal arguments does not get reflected in the actual argument. This way of passing variable is known as call by value.

call by reference – In this way of passing variable, a reference to the object in memory is passed. Both the formal arguments and the actual arguments (variables in the calling code) refer to the same object. Hence, any changes in formal arguments does get reflected in the actual argument.

Python uses pass by reference mechanism. As variable in Python is a label or reference to the object in the memory, both the variables used as actual argument as well as formal arguments really refer to the same object in the memory. We can verify this fact by checking the `id()` of the passed variable before and after passing.

```
def testfunction(arg):
    print ("ID inside the function:", id(arg))
    var = "Hello"
    print ("ID before passing:", id(var))
    testfunction(var)
```

Function Parameters

A function with parameters is defined to accept one or more inputs, called parameters, which act as placeholders for the data passed to the function when it is called. These parameters allow the function to perform operations based on the provided inputs.

Parameters are variables that are defined in the function definition. They are assigned the values which were passed as arguments when the function was called, elsewhere in the code.

Syntax:

```
def function_name(parameter1, parameter2, ...):
    # Function body (code block)
    # Use parameters in operations
    return result # Optional
```

Example:

```
def greet(name):
    message = f"Hello, {name}!"
    return message
# Calling the function
print(greet("Alice")) # Output: Hello, Alice!
print(greet("Bob"))   # Output: Hello, Bob!
```

Python Function Arguments

Function arguments are the values or variables passed into a function when it is called. The behavior of a function often depends on the arguments passed to it.

While defining a function, you specify a list of variables (known as formal parameters) within the parentheses. These parameters act as placeholders for the data that will be passed to the function when it is called. When the function is called, value to each of the formal arguments must be provided. Those are called actual arguments.

Formal and Actual Arguments

Formal Arguments (Parameters)

Formal arguments are the variables listed in a function's definition. They act as placeholders that receive values when the function is called. These are also called **parameters**.

They define the input structure of the function and are used within the function to perform computations or operations.

Characteristics:

- Formal arguments are local to the function's scope, meaning they cannot be accessed outside the function.
- They are defined in the function signature, e.g., `def function_name(param1, param2):`.
- The number and type of formal arguments determine how the function must be called.

Example:

```
def multiply(x, y): # 'x' and 'y' are formal arguments
    return x * y
```

Here, x and y are formal arguments that expect two values when the function is called.

Edge Case:

If a formal argument is defined but not provided during the function call (and has no default value), Python raises a `TypeError`.

```
multiply(5) # TypeError: multiply() missing 1 required positional argument: 'y'
```

Actual Arguments

Actual arguments are the real values or expressions passed to a function during its invocation. They are mapped to the formal arguments.

They provide the data that the function will process.

Characteristics:

- Actual arguments can be literals (e.g., 5, "Hello"), variables, or expressions.
- They must match the number and type expected by the formal arguments unless defaults or variable-length arguments are used.

Example:

```
result = multiply(4, 5) # '4' and '5' are actual arguments
print(result) # Output: 20
```

Here, 4 is assigned to x, and 5 is assigned to y.

Edge Case:

- Passing too many actual arguments causes a `TypeError` unless the function supports variable-length arguments (`*args`).

```
multiply(4, 5, 6) # TypeError: multiply() takes 2 positional
arguments but 3 were given
```

Positional Arguments

Positional arguments are arguments passed to a function in the same order as the formal parameters are defined in the function signature.

They are the simplest way to pass arguments, relying on the order of parameters to map values correctly.

- **Characteristics:**
 - The position of each actual argument corresponds to the position of the formal parameter.
 - They are mandatory unless the parameter has a default value.

Example:

```
def student_info(name, age, grade):
    print(f"Name: {name}, Age: {age}, Grade: {grade}")
student_info("Alice", 15, "10th") # Output: Name: Alice, Age: 15, Grade: 10th
```

Here, "Alice" maps to name, 15 to age, and "10th" to grade based on position.

- **Edge Case:**
 - If the number of positional arguments doesn't match the number of formal parameters, Python raises a `TypeError`.

```
student_info("Alice", 15) # TypeError: student_info() missing 1 required
positional argument: 'grade'
```

Keyword Arguments

Keyword arguments are passed to a function by explicitly specifying the parameter name and its value (e.g., `param=value`) during the function call.

They improve code readability and allow arguments to be passed in any order, avoiding errors due to position mismatches.

- **Characteristics:**
 - The parameter name is used to map the value, making the order irrelevant.
 - They are often used when a function has many parameters or when some arguments are optional.
- **Example:**

```
def book_details(title, author, year):
    print(f"Book: {title}, Author: {author}, Year: {year}")

book_details(author="J.K. Rowling", title="Harry Potter", year=1997)
# Output: Book: Harry Potter, Author: J.K. Rowling, Year: 1997
```

- **Edge Case:**
 - If a keyword argument is passed for a non-existent parameter, Python raises a `TypeError`.

```
book_details(title="Harry Potter", writer="J.K. Rowling") # TypeError: book_details() got an unexpected keyword argument 'writer'
```

- **Advantages:**
 - Enhances readability, especially for functions with many parameters.
 - Reduces errors caused by incorrect argument order.
 - Useful in libraries or APIs where function signatures are complex.

Default Arguments in Python

- Default arguments are values that are provided while defining functions.
- The assignment operator `=` is used to assign a default value to the argument.
- Default arguments become optional during the function calls.
- If we provide a value to the default arguments during function calls, it overrides the default value.
- The function can have any number of default arguments.
- Default arguments should follow non-default arguments.

In the below example, the default value is given to argument `b` and `c`.

```
def add(a,b=5,c=10):
    return (a+b+c)
```

This function can be called in one of three ways:

Giving Only the Mandatory Argument

```
print(add(3))
#Output:18
```

Giving One of the Optional Arguments

3 is assigned to `a`, 4 is assigned to `b`.

```
print(add(3,4))
#Output:17
```

Giving All the Arguments

```
print(add(2,3,4))
#Output:9
```

Default values are evaluated only once at the point of the function definition in the defining scope. So, it makes a difference when we pass mutable objects like a list or dictionary as default values.

Variable Length Arguments

Variable-length arguments allow a function to accept an arbitrary number of arguments, either as positional arguments (`*args`) or keyword arguments (`**kwargs`).

They provide flexibility when the number of arguments is unknown or varies.

- **Types:**
 - ***args:** Collects extra positional arguments into a tuple.
 - ****kwargs:** Collects extra keyword arguments into a dictionary.

Non-Keyword Variable-Length Arguments (*args)

The *args syntax allows a function to accept any number of positional arguments, which are stored in a tuple named args (or any name preceded by *).

Characteristics:

- The * unpacks the arguments into a tuple.
- Useful for functions that need to handle a variable number of inputs, like summing numbers or concatenating strings.

Example:

```
def average(*args):
    if not args: # Check if args is empty
        return 0
    return sum(args) / len(args)

print(average(10, 20, 30)) # Output: 20.0
print(average(1, 2, 3, 4, 5)) # Output: 3.0
print(average()) # Output: 0
```

- **Edge Case:**
 - If *args is used, it must appear **after regular and default parameters** in the function definition.

```
def func(a, *args, b): # SyntaxError: *args must come after all
    positional parameters
    pass
```

Keyword Variable-Length Arguments (kwargs)**

The **kwargs syntax allows a function to accept any number of keyword arguments, which are stored in a dictionary named kwargs (or any name preceded by **).

Characteristics:

- The ** unpacks the keyword arguments into a dictionary where keys are parameter names and values are the arguments.
- Useful for passing configuration options or metadata.

Example:

```
def user_profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

user_profile(name="Alice", age=25, city="London", job="Engineer")
```

- **Combining All Argument Types:**
 - You can combine regular, default, *args, and **kwargs in a function, but they must follow this order:
 1. Regular positional parameters

2. Default parameters
3. *args
4. **kwargs

Example:

```
def complex_function(a, b=10, *args, **kwargs):
    print(f"Regular: {a}")
    print(f"Default: {b}")
    print(f"Positional extras: {args}")
    print(f"Keyword extras: {kwargs}")

complex_function(1, 2, 3, 4, x="extra", y="data")
```

Return Values and `return` Statements in Python

In Python, **return values** and the **return statement** are fundamental parts of defining and using functions. They allow a function to send data back to the part of the program where it was called. This is useful when a function performs a computation or task and needs to return a result.

What is a Return Value?

A **return value** is the output that a function sends back to the caller after execution. This output can be:

- A single value like a number or string
- Complex types such as lists, dictionaries, or even other functions
- A group of values returned together as a tuple

For example:

```
def add(a, b):
    return a + b

result = add(3, 4) # result now holds the value 7
```

In this example, `add()` returns the sum of `a` and `b`, which is then stored in the variable `result`.

What is a `return` Statement?

The **return statement** is used inside a function to:

- Exit the function immediately
- Pass the specified value back to the caller

Once a `return` statement is executed, no further code in the function is executed. This makes `return` useful for exiting a function early, especially based on a condition.

Syntax of `return` Statement

```
def function_name(parameters):
    # Function body
    return value
```

You can store the returned result like this:

```
variable = function_name(arguments)
```

Key Concepts to Remember

- **Single Return Value:**

A function can return a single result (like `return x + y`).

- **Multiple Return Values:**

A function can return multiple values by separating them with commas. Python automatically packs these into a tuple.

Example:

```
return x, y # returns a tuple (x, y)
```

- **No Return Statement:**

If a function doesn't have a `return` statement, Python automatically returns `None`.

Example:

```
def greet():  
    print("Hello!") # No return value
```

- **Early Exit:**

The `return` statement can be used to stop function execution early.

Example:

```
def check_even(n):  
    if n % 2 == 0:  
        return True  
    return False
```

Why `return` Is Important

The use of `return` makes functions more useful and modular. By returning values, functions can pass results around in a program, making code easier to manage, reuse, and test.

The None Value in Python

`None` is a **special constant** in Python that represents the **absence of a value** or a **null value**. It is commonly used when a variable or function does not hold any meaningful value or result. `None` is an object of the `NoneType` class and is the **only instance** of that class in Python.

Characteristics of None

1. Singleton Object

- Python has only **one None object**, and it is **immutable** (cannot be changed).
- Because there is only one `None`, it is safe to compare with `is` (identity check) instead of `==`.

```
x = None  
if x is None:  
    print("x has no value")
```

2. Default Return Value

- If a function does **not explicitly return** a value using a `return` statement, Python **automatically returns None**.

```
def greet():  
    print("Hello")
```



```
result = greet()      # prints "Hello"
print(result)         # prints None
```

3. Placeholder Use

- `None` is often used as a **placeholder** to indicate that a variable exists but has not been assigned a value yet.
- This is useful in function defaults and when building data structures gradually.

```
def show_message(msg=None):
    if msg is None:
        print("No message provided.")
    else:
        print("Message:", msg)
```

4. Boolean Context

- `None` is considered **falsy** in Python. This means it behaves like `False` in conditional checks.

```
if not None:
    print("None is treated as False")
```

When to Use None

- As a **default return value** when there's nothing to return from a function.
- As a **placeholder** for optional parameters or uninitialized variables.
- In **comparisons** to detect the absence of a value (always use `is` instead of `==`).
- To signal special cases like errors, missing data, or "do nothing" behavior.

Remember

- `None` is **not the same** as `0`, `False`, an empty string `''`, or an empty list `[]`. All of those represent *something*, whereas `None` represents *nothing*.
- `None` is **not printed** unless you explicitly print it or store it in a variable.

Understanding Local and Global Scope in Python

In programming, **scope** refers to the context in which a variable or name is defined and accessed. It tells you **where a variable can be used** and **how long it exists** in memory. Python has well-defined rules for variable scope to help you manage data safely and predictably.

There are two main types of scope in Python:

◆ 1. Global Scope

A **global variable** is a variable that is defined **outside of all functions** and is accessible throughout the file or module. These variables can be used **inside functions** (read-only by default) and **outside functions** (read and write).

Characteristics:

- Created outside of all functions.

- Accessible from **any part of the code**, including inside functions.
- If you want to **modify** it inside a function, you must declare it with the `global` keyword.

Example: Accessing a Global Variable

```
x = 10 # global variable

def show():
    print("Inside function:", x)

show()
print("Outside function:", x)
```

Output:

```
Inside function: 10
Outside function: 10
```

Here, `x` is accessible both inside and outside the function because it is in the global scope.

Example: Modifying a Global Variable Inside a Function

```
x = 5

def update():
    global x # tells Python to use the global variable x
    x = 20

update()
print(x) # Output: 20
```

If you don't use `global`, Python will treat `x` inside the function as a **new local variable**, leaving the global `x` unchanged.

◆ 2. Local Scope

A **local variable** is one that is **defined inside a function** and only exists **within that function**. It is created when the function starts and destroyed when the function ends.

Characteristics:

- Created and used **only inside** the function where it's defined.
- **Cannot** be accessed outside the function.
- Takes priority over any global variable with the same name (local shadowing).

Example: Using a Local Variable

```
def greet():
    name = "Aadil" # local variable
    print("Hello", name)

greet()
print(name) # ✗ Error: name is not defined
```

Here, `name` only exists **inside** the function. Once `greet()` is done, the variable is **destroyed**.

Example: Local Variable Shadows Global Variable

```
x = "Global"
```

```
def test():
    x = "Local"
    print("Inside function:", x)

test()
print("Outside function:", x)
```

Output:

```
Inside function: Local
Outside function: Global
```

The local variable `x` **overrides** the global `x` **within the function**. Outside the function, the global variable is still used.

The global Statement in Python

In Python, the `global` statement is used **inside a function** to **refer to a variable that is defined in the global scope**. Without this keyword, any assignment to a variable inside a function will by default create a **new local variable**, even if a variable with the same name exists globally.

The `global` keyword tells Python not to create a new local variable, but instead to use and modify the globally defined variable.

Why Do We Need global?

By default, variables defined inside a function are **local to that function**. This means you can read global variables inside functions, but if you try to **modify** them, Python will treat them as **local unless you use global**.

Example Without global (local variable created):

```
x = 10

def change():
    x = 20    # creates a new local variable, doesn't
              # affect global x
    print("Inside function:", x)

change()
print("Outside function:", x)
```

Output:

```
Inside function: 20
Outside function: 10
```

Here, `x = 20` inside the function creates a **new local variable** named `x`. The global `x` remains unchanged.

Example With global (global variable modified):

```
x = 10

def change():
    global x
    x = 20    # modifies the global variable
    print("Inside function:", x)

change()
print("Outside function:", x)
```

Output:

```
Inside function: 20
Outside function: 20
```

Now, `global x` tells Python to use the `x` defined outside the function, so the change affects both inside and outside values.

✦ Syntax of `global`

```
global variable_name
```

You can even use it for **multiple variables**:

```
global a, b, c
```

Important Points

- The `global` statement must appear **before the variable is used or assigned** in the function.
- You **cannot use `global` to declare new variables only inside a function**—it's meant to **refer to variables that exist in the global scope**.
- If the global variable doesn't already exist, using `global` will **create it in the global namespace**.

✗ When You Should Avoid `global`

While `global` can be helpful, it's often **discouraged in large programs** because:

- It makes debugging harder (global state can be changed unexpectedly).
- It reduces the **modularity** of functions.
- It can lead to **side effects** and unpredictable behavior.

Instead, consider using **function parameters**, **return values**, or **classes/objects** to manage state.

SLICING IN PYTHON

Slicing in Python is a technique used to access a sub-sequence of a sequence, such as a list, tuple, or string. It allows you to extract specific elements from a sequence by specifying a starting and ending index, and optionally a step value. This is done using the colon `:` operator within square brackets `[]`.

Basic Syntax

The basic syntax for slicing is:

```
sequence[start:end:step]
```

start: The index of the first element in the sub-sequence.

end: The index of the last element in the sub-sequence (excluding the element at the end index).

step: The interval between elements in the sub-sequence (optional).

Examples

Extracting Substrings from a String

Suppose we have a string and we want to extract specific words from it:

```
sentence = "The quick brown fox jumps over the lazy dog"
first_word = sentence[:3]
second_word = sentence[4:9]
third_word = sentence[10:15]

print(first_word) # Output: "The"
print(second_word) # Output: "quick"
print(third_word) # Output: "brown"
```

In this example, we use slicing to extract the first, second, and third words from the sentence.

Filtering a List

We can use slicing to filter elements from a list. For instance, to extract all odd numbers from a list:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
odd_numbers = numbers[::2]
print(odd_numbers) # Output: [1, 3, 5, 7, 9]
```

Here, the `::2` slice means selecting every other element starting from the first element^{[1](#)}.

Modifying Parts of a List

Slicing can also be used to modify parts of a list:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
numbers[1:4] = [10, 20, 30]
print(numbers) # Output: [1, 10, 20, 30, 5, 6, 7, 8, 9]
```

In this example, we replace the elements at indices 1 to 3 with new values^{[2](#)}.

Using the `slice()` Function

The `slice()` function can be used to create a slice object, which can then be used to slice a sequence:

```
a = ("a", "b", "c", "d", "e", "f", "g", "h")
x = slice(2)
print(a[x]) # Output: ('a', 'b')
```

This example demonstrates how to use the `slice()` function to get the first two items of a tuple^{[3](#)}.

Extracting a Sublist:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
sublist = numbers[2:6] # Extract elements from index 2 to 5
print(sublist)
# Output: [3, 4, 5, 6]
```

Reversing a List:

```
fruits = ["apple", "banana", "cherry"]
reversed_fruits = fruits[::-1] # Reverse the list
print(reversed_fruits)
# Output: ['cherry', 'banana', 'apple']
```

Copying a List:

```
original = [1, 2, 3]
copy = original[:] # Create a shallow copy
copy.append(4)

print(original)
# Output: [1, 2, 3]
```

```
print(copy)
# Output: [1, 2, 3, 4]
```

Skipping Elements:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
skip_every_second = numbers[::2] # Skip every second element
print(skip_every_second)
# Output: [1, 3, 5, 7, 9]
```

Conclusion

Slicing is a powerful tool in Python that allows you to efficiently access and manipulate sub-sequences of data. By understanding and using slicing effectively, you can simplify many tasks in Python programming, such as extracting substrings, filtering lists, and modifying data.

Built in functions used in lists in python

1. len(): Count the Number of Elements

Purpose:Returns the total number of items present in a list.

Example:

```
fruits = ['apple', 'banana', 'cherry']
print(len(fruits)) # Output: 3
```

The list has 3 elements, so len() returns 3.

2. max(): Find the Maximum Value

Purpose:Returns the highest value from the list.

Example:

```
marks = [45, 67, 89, 23]
print(max(marks)) # Output: 89
```

Here, 89 is the highest number in the list.

3. min(): Find the Minimum Value

Purpose:Returns the lowest value from the list.

Example:

```
marks = [45, 67, 89, 23]
print(min(marks)) # Output: 23
```

4. sum(): Total Sum of Elements

Purpose:Adds all the numerical values in the list and returns the total.

Example:

```
numbers = [1, 2, 3, 4]
print(sum(numbers)) # Output: 10
```

5. sorted(): Sort the List

Purpose: Returns a **new sorted list** without modifying the original.

Example:

```
names = ['john', 'anna', 'zara']
print(sorted(names)) # Output: ['anna', 'john', 'zara']
```

6. any(): Check if Any Value is True

Purpose: Returns `True` if **at least one** item in the list is considered `True` (non-zero, non-empty), otherwise `False`.

Example:

```
items = [0, '', False, 10]
print(any(items)) # Output: True
```

Because 10 is truthy, the result is `True`.

7. all(): Check if All Values are True

Purpose: Returns `True` only if **all items** in the list are truthy.

Example:

```
items = [1, True, 5]
print(all(items)) # Output: True

items = [1, 0, 3]
print(all(items)) # Output: False
```

8. enumerate(): Get Index with Value

Purpose: Used to loop through a list with both **index and value**.

Example:

```
colors = ['red', 'green', 'blue']
for index, color in enumerate(colors):
    print(index, color)

# Output:
# 0 red
# 1 green
# 2 blue
```

9. map(): Apply a Function to All Elements

Purpose:Returns a new list with a function applied to every item.

Example:

```
numbers = [1, 2, 3]
squares = list(map(lambda x: x*x, numbers))
print(squares)  # Output: [1, 4, 9]
```

10. filter(): Filter Items by Condition

Purpose:Returns a new list with only items that match a condition.

Example:

```
numbers = [1, 2, 3, 4, 5]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # Output: [2, 4]
```

11. zip(): Combine Two Lists

Purpose:Merges two (or more) lists into a list of tuples, pairing elements with the same index.

Example:

```
names = ['Alice', 'Bob']
scores = [90, 85]
result = list(zip(names, scores))
print(result)  # Output: [('Alice', 90), ('Bob', 85)]
```

12. reversed(): Reverse a List

Purpose:Returns a reversed version of the list (as an iterator).

Example:

```
nums = [1, 2, 3]
print(list(reversed(nums)))  # Output: [3, 2, 1]
```

13. list(): Convert to a List

Purpose:Converts other iterables (like strings, tuples) into a list.

Example:

```
text = "abc"
print(list(text))  # Output: ['a', 'b', 'c']

tup = (1, 2, 3)
print(list(tup))  # Output: [1, 2, 3]
```


List Comprehension in Python

List comprehension in Python is a concise and elegant way to create lists. It allows you to generate a new list by applying an expression to each item in an iterable (like a list, range, or string) and optionally filtering items with a condition. It is often used to make code more readable and compact. This helps us to write cleaner, more readable code compared to traditional looping techniques.

Syntax of List Comprehension

[expression for item in iterable if condition]

- **expression**: The operation or value to include in the new list.
- **item**: The variable representing each element in the iterable.
- **iterable**: The source of data (e.g., list, range, etc.).
- **condition** (optional): A filter to include only certain items.

Example:

```
numbers = [1, 2, 3, 4, 5]

squares = [x**2 for x in numbers] # Output: [1, 4, 9, 16, 25]
```

For Loop vs. List Comprehension

The main difference is that a **for loop** requires multiple lines to create a new list by iterating over items and manually adding each one. Whereas, **list comprehension** do the same task in a single line, this makes the code simpler and easier to read.

```
numbers = []
for x in range(5):
    numbers.append(x)
print(numbers)

List comprehension
numbers = [x for x in range(5)]
print(numbers)
```

Nested List Comprehensions

You can perform nested iterations within a list comprehension to handle more complex cases, such as flattening a list of lists:

```
matrix = [[1, 2], [3, 4], [5, 6]]

flat_list = [num for row in matrix for num in row]

# Output: [1, 2, 3, 4, 5, 6]
```

Variations of List Comprehension

In Python, **list comprehension** has several powerful variations that allow you to create lists in flexible and efficient ways. These variations include:

List Comprehension with if Condition (Filtering)

Include only items that meet a condition.

```
even_numbers = [x for x in range(10) if x % 2 == 0]
print(even_numbers)  # Output: [0, 2, 4, 6, 8]
```

List Comprehension with if-else (Conditional Expression)

Include logic inside the expression using if-else.

```
labels = ["even" if x % 2 == 0 else "odd" for x in range(5)]
print(labels)  # Output: ['even', 'odd', 'even', 'odd', 'even']
```

Nested List Comprehension

Used when working with nested structures like 2D lists or matrices.

```
matrix = [[1, 2], [3, 4], [5, 6]]
flattened = [num for row in matrix for num in row]
print(flattened)  # Output: [1, 2, 3, 4, 5, 6]
```

List Comprehension with Multiple for Loops

Generate combinations or iterate over multiple sequences.

```
pairs = [(x, y) for x in [1, 2] for y in [3, 4]]
print(pairs)  # Output: [(1, 3), (1, 4), (2, 3), (2, 4)]
```

List Comprehension with Function Calls

You can call functions inside the comprehension expression.

```
def square(n):
    return n * n

result = [square(x) for x in range(5)]
print(result)  # Output: [0, 1, 4, 9, 16]
```

List Comprehension on Strings

Apply operations on characters in strings.

```
uppercase = [char.upper() for char in "hello"]
print(uppercase)  # Output: ['H', 'E', 'L', 'L', 'O']
```

Advantages of List Comprehension

1. **Conciseness:** List comprehensions allow you to create lists in a single line of code, as opposed to multiple lines needed with for loops.
2. **Readability:** They are often more readable and thus easier to understand at a glance, especially for simple transformations.

3. Performance: List comprehensions can be faster than using traditional for loops since they are optimized internally by Python's interpreter.

When Not to Use List Comprehension

While list comprehensions are powerful, they can become less readable with complex conditions or nested loops. In such cases, traditional for loops or separate functions may be clearer.

The Dictionary Data Type

1. What is a Dictionary?

A dictionary in Python is a mutable, unordered collection of key-value pairs. Each key is unique and maps to a specific value. Dictionaries are highly efficient for retrieving data using keys and are defined using curly braces {} or the dict() constructor.

- **Key Characteristics:**

- Keys must be immutable (e.g., strings, numbers, tuples) and unique.
- Values can be of any data type (e.g., strings, numbers, lists, or even other dictionaries).
- Dictionaries are mutable, meaning you can change their contents after creation.
- They are unordered (in Python 3.7+, they maintain insertion order, but this is not the same as being sorted).

Example:

```
# A dictionary storing student names and their marks

student_marks = {"Alice": 85, "Bob": 90, "Charlie": 78}
```

2. Creating a Dictionary

You can create a dictionary in several ways:

- **Using curly braces {}:**

```
my_dict = {"name": "Alice", "age": 20, "city": "New York"}
```

- **Using the dict() constructor:**

```
my_dict = dict(name="Alice", age=20, city="New York")
```

- **Using a list of tuples:**

```
my_dict = dict([("name", "Alice"), ("age", 20), ("city", "New York")])
```

- **Empty dictionary:**

```
empty_dict = {}
```

Exam Tip: Mention that dictionaries are ideal for storing data where you need quick lookups using unique keys, like a phone book mapping names to numbers.

3. Accessing and Modifying Key-Value Pairs

- **Accessing Values:** Use the key inside square brackets [] or the get() method to access a value.

```
student_marks = {"Alice": 85, "Bob": 90}
```

```
print(student_marks["Alice"]) # Output: 85
```

```
print(student_marks.get("Bob")) # Output: 90
```

- The `get()` method is safer because it returns `None` (or a default value) if the key doesn't exist, whereas `[]` raises a `KeyError`.

- **Modifying Values:** Assign a new value to an existing key or add a new key-value pair.

```
student_marks["Alice"] = 88 # Update Alice's marks
```

```
student_marks["David"] = 95 # Add a new key-value pair
```

```
print(student_marks) # Output: {'Alice': 88, 'Bob': 90, 'David': 95}
```

- **Deleting Key-Value Pairs:** Use the `del` statement or `pop()` method.

```
del student_marks["Bob"] # Remove Bob's entry
```

```
print(student_marks) # Output: {'Alice': 88, 'David': 95}
```

```
student_marks.pop("Alice") # Remove Alice and return her value
```

```
print(student_marks) # Output: {'David': 95}
```

Example: Imagine a dictionary as a phone book. The name (key) helps you find a phone number (value). You can update a number, add a new contact, or remove one.

4. Built-In Functions Used on Dictionaries

Python provides several built-in functions to work with dictionaries:

- **len():** Returns the number of key-value pairs.

```
my_dict = {"a": 1, "b": 2, "c": 3}
```

```
print(len(my_dict)) # Output: 3
```

- **in Operator:** Checks if a key exists in the dictionary.

```
print("a" in my_dict) # Output: True
```

```
print("d" in my_dict) # Output: False
```

- **dict():** Creates a dictionary (already covered in creation).
- **sorted():** Returns a sorted list of keys.

```
print(sorted(my_dict)) # Output: ['a', 'b', 'c']
```

Exam Tip: Explain that `in` checks keys, not values, unless explicitly coded to check values (e.g., `value in my_dict.values()`).

5. Dictionary Methods

Dictionaries come with useful methods to manipulate and retrieve data:

- **get(key, default):** Returns the value for a key or a default value if the key is not found.

```
print(my_dict.get("d", 0)) # Output: 0 (since "d" doesn't exist)
```

- **keys():** Returns a view of all keys.

```
print(my_dict.keys()) # Output: dict_keys(['a', 'b', 'c'])
```

- **values():** Returns a view of all values.

```
print(my_dict.values()) # Output: dict_values([1, 2, 3])
```

- **items():** Returns a view of key-value pairs as tuples.

```
print(my_dict.items()) # Output: dict_items([('a', 1), ('b', 2), ('c', 3)])
```

- **update(other_dict):** Updates the dictionary with key-value pairs from another dictionary or iterable.

```
my_dict.update({"b": 5, "d": 4})
```

```
print(my_dict) # Output: {'a': 1, 'b': 5, 'c': 3, 'd': 4}
```

- **pop(key, default):** Removes and returns the value for a key, or a default value if the key is not found.

```
print(my_dict.pop("c")) # Output: 3
```

- **popitem():** Removes and returns the last inserted key-value pair as a tuple.

```
print(my_dict.popitem()) # Output: ('d', 4)
```

- **clear():** Removes all key-value pairs.

```
my_dict.clear()
```

```
print(my_dict) # Output: {}
```

Example: Think of a dictionary as a student record system. You can use `get()` to check a student's grade without crashing if they're not in the system, `keys()` to list all students, or `update()` to add new student records.

Exam Tip: For each method, provide a one-line description and an example. Mention that methods like `keys()`, `values()`, and `items()` return views, which dynamically reflect changes to the dictionary.

Tuples

1. What is a Tuple?

A tuple is an immutable, ordered collection of elements in Python, defined using parentheses () or the tuple() constructor. Once created, you cannot change a tuple's elements (no additions, deletions, or modifications).

- **Key Characteristics:**
 - Immutable: Elements cannot be changed after creation.
 - Ordered: Elements maintain their insertion order.
 - Can contain mixed data types (e.g., numbers, strings, lists).
 - Useful for storing fixed data, like coordinates or days of the week.

Example:

```
my_tuple = (1, "hello", 3.14)
```

2. Creating Tuples

- **Using parentheses:**

```
my_tuple = (1, 2, 3)
```

- **Using the tuple() constructor:**

```
my_tuple = tuple([1, 2, 3])
```

- **Single-element tuple** (requires a trailing comma):

```
single_tuple = (5,) # Without comma, (5) is just an integer
```

- **Empty tuple:**

```
empty_tuple = ()
```

Example: A tuple can represent a point in 2D space, like (3, 4), where the values are fixed and shouldn't change.

3. Basic Tuple Operations

- **Concatenation (+):** Combines two tuples.

```
tuple1 = (1, 2)
```

```
tuple2 = (3, 4)
```

```
print(tuple1 + tuple2) # Output: (1, 2, 3, 4)
```

- **Repetition (*):** Repeats a tuple.

```
print(tuple1 * 2) # Output: (1, 2, 1, 2)
```

- **Membership Testing (in):** Checks if an element exists in a tuple.

```
print(2 in tuple1)  # Output: True
```

Exam Tip: Emphasize that tuples are immutable, so operations like concatenation create new tuples rather than modifying the original.

4. Indexing and Slicing in Tuples

- **Indexing:** Access elements using their position (0-based index).

```
my_tuple = ("apple", "banana", "cherry")
```

```
print(my_tuple[1])  # Output: banana
```

```
print(my_tuple[-1])  # Output: cherry (negative indexing starts from the end)
```

- **Slicing:** Extract a portion of the tuple using [start:end:step].

```
print(my_tuple[0:2])  # Output: ('apple', 'banana')
```

```
print(my_tuple[::2])  # Output: ('apple', 'cherry') (every second element)
```

Example: Think of a tuple as a fixed list of days: ("Mon", "Tue", "Wed"). You can access "Tue" with index 1 or slice to get ("Mon", "Tue").

5. Built-In Functions Used on Tuples

- **len():** Returns the number of elements.

```
my_tuple = (1, 2, 3)
```

```
print(len(my_tuple))  # Output: 3
```

- **min() and max():** Returns the smallest or largest element (works if elements are comparable).

```
print(min((1, 2, 3)))  # Output: 1
```

```
print(max((1, 2, 3)))  # Output: 3
```

- **sum():** Sums elements (works for numeric tuples).

```
print(sum((1, 2, 3)))  # Output: 6
```

- **sorted():** Returns a sorted list of tuple elements.

```
print(sorted((3, 1, 2)))  # Output: [1, 2, 3] (returns a list, not a tuple)
```

- **in Operator:** Checks if an element exists.


```
print(2 in (1, 2, 3)) # Output: True
```

Exam Tip: Clarify that `sorted()` returns a list, not a tuple, and that `min()`, `max()`, and `sum()` require compatible data types.

Sets

1. What is a Set?

A set is a mutable, unordered collection of unique elements in Python, defined using curly braces `{}` or the `set()` constructor. Sets are useful for storing non-duplicate items and performing mathematical set operations like union and intersection.

- **Key Characteristics:**

- Unordered: Elements have no specific order.
- Unique: No duplicate elements are allowed.
- Mutable: You can add or remove elements (but elements themselves must be immutable).
- Commonly used for membership testing and set operations.

Example:

```
my_set = {1, 2, 3, 3} # Duplicates are removed
```

```
print(my_set) # Output: {1, 2, 3}
```

2. Creating Sets

- **Using curly braces:**

```
my_set = {1, 2, 3}
```

- **Using the `set()` constructor:**

```
my_set = set([1, 2, 3, 3]) # Duplicates removed
```

- **Empty set** (use `set()`, not `{}`, as `{}` creates an empty dictionary):

```
empty_set = set()
```

Example: A set can represent unique student IDs in a class, ensuring no duplicates.

3. Basic Set Operations

- **Adding Elements:** Use `add()` to add a single element.

```
my_set = {1, 2}
```

```
my_set.add(3)
```

```
print(my_set) # Output: {1, 2, 3}
```

- **Removing Elements:** Use `remove()` (raises `KeyError` if element not found) or `discard()` (no error if element not found).

```
my_set.remove(2) # Output: {1, 3}
```

```
my_set.discard(4) # No error even if 4 is not in set
```

4. Set Methods

- **add(element):** Adds an element to the set.

```
my_set.add(4) # Output: {1, 3, 4}
```

- **remove(element):** Removes an element (raises `KeyError` if not found).

```
my_set.remove(3) # Output: {1, 4}
```

- **discard(element):** Removes an element (no error if not found).

```
my_set.discard(5) # No error
```

- **pop():** Removes and returns a random element (since sets are unordered).

```
print(my_set.pop()) # Output: 1 (or another element)
```

- **clear():** Removes all elements.

```
my_set.clear() # Output: set()
```

- **Set Operations:**
 - **union() or |:** Combines elements from two sets.

```
set1 = {1, 2}
```

```
set2 = {2, 3}
```

```
print(set1.union(set2)) # Output: {1, 2, 3}
```

```
print(set1 | set2) # Output: {1, 2, 3}
```

- **intersection() or &:** Returns common elements.

```
print(set1.intersection(set2)) # Output: {2}
```

```
print(set1 & set2) # Output: {2}
```

- **difference() or -:** Returns elements in the first set but not in the second.

```
print(set1.difference(set2)) # Output: {1}
```

```
print(set1 - set2) # Output: {1}
```

- **symmetric_difference() or ^**: Returns elements in either set but not both.

```
print(set1.symmetric_difference(set2)) # Output: {1, 3}
```

```
print(set1 ^ set2) # Output: {1, 3}
```

Example: Think of sets as two groups of students taking different courses. You can use `union()` to find all students, `intersection()` to find students taking both courses, or `difference()` to find students in one course but not the other.

5. Traversing Sets

Sets can be traversed using loops since they are iterable. Common ways include:

- **Using a for loop:**

```
my_set = {1, 2, 3}
```

```
for item in my_set:
```

```
    print(item) # Output: 1, 2, 3 (order not guaranteed)
```

- **Using set comprehension:**

```
squared_set = {x**2 for x in my_set}
```

```
print(squared_set) # Output: {1, 4, 9}
```

Exam Tip: Highlight that sets are unordered, so the traversal order is not predictable. Also, mention that sets are iterable but don't support indexing or slicing due to their unordered nature.